Informe de practica: Recuperacion aumentada por generacion (RAG) con GROQ sobre un documento PDF en ingles

Proyecto: Sistema de recuperacion aumentada por generacion (RAG) con GROQ sobre un documento PDF en ingles

Notebook de origen: https://colab.research.google.com/drive/1iCONlz-J781Rm6JT5QbiTpdc7MojoerN?usp=sharing

Repositorio: https://github.com/EnriqueDiaz2/LLM-Rag.git

Fecha: 30 de octubre de 2026

Indice

1. Resumen
2. Contexto del proyecto
3. Evidencia de la cuenta en GROQ y de la API KEY
4. Metodologia del RAG sobre el documento PDF
5. Resultados del punto 3: comparacion de busquedas con llama3.1:8b
6. Resultados del punto 4: nuevo modelo LLM en GROQ
7. Enlaces
8. Conclusiones personales
9. Referencias

Resumen

Este informe documenta el desarrollo de un sistema de **generacion aumentada por recuperacion (RAG)** implementado sobre un **documento PDF en ingles** y ejecutado a traves de la API de **GROQ**.

El estudiante debe de usar un documento PDF en ingles para vectorizar el texto de dicho documento.

Para ello se crea una cuenta en la plataforma **GROQ**, se obtiene una **API KEY** y se configura

el acceso a los modelos de embeddings y de chat.

En el ejercicio se utilizo el modelo de chat **llama3.1:8b** para responder las consultas del

usuario a partir del contexto recuperado.

Solo se utilizo el resultado de la **busqueda hibrida** para generar las respuestas.

Con las mismas **3 consultas** distintas se repitio el proceso con la **busqueda densa** y con la

busqueda dispersa, y finalmente se compararon los resultados obtenidos.

El punto 4 consta de identificar otro modelo LLM dentro de la plataforma **GROQ** y repetir el

punto 3 con el nuevo modelo.

1. Contexto del proyecto

1.1 La tarea

El estudiante debe de:

10. Usar un **documento PDF en ingles** para vectorizar el texto de dicho documento.
11. Crear una cuenta en **GROQ** y crear una **API KEY**.
12. En el ejercicio actual, solo se utiliza el resultado de la **busqueda hibrida** para que el modelo **llama3.1:8b** de respuesta a la consulta del usuario.
13. Realizar **3 consultas diferentes** sobre el documento con el resultado de la busqueda **hibrida**.
14. Aplicar el modelo LLM en el resultado de la **busqueda densa** con las mismas **3 consultas**.
15. Aplicar el modelo LLM en el resultado de la **busqueda dispersa** con las mismas **3 consultas**.
16. Comparar los resultados obtenidos al usar cada una de las **3 busquedas** (densa, dispersa e hibrida).

1.2 Arquitectura del sistema RAG

#	Etapa	Tecnologia / componente
1	Carga del documento PDF en ingles	`pypdf`
2	Extraccion del texto	`pypdf`
3	Fragmentacion en chunks	Funcion `dividir_en_chunks`
4	Embeddings de los chunks	GROQ, modelo `embed-english-v3.0`
5	Busqueda densa	Similitud coseno
6	Busqueda dispersa	BM25
7	Fusión de busquedas	Reciprocal Rank Fusion (RRF)
8	Generación de respuesta	GROQ chat completions, modelo `llama3.1:8b`

2. Evidencia de la cuenta en GROQ y de la API KEY

Para el desarrollo del sistema RAG fue necesario tener acceso a la plataforma de inferencia **GROQ** y disponer de una **API KEY** valida.

Elemento	Evidencia
Cuenta creada en GROQ	https://groq.com/
API KEY	Generada en el dashboard de GROQ (el valor completo no se incluye en este informe por seguridad)
Endpoint de embeddings	https://api.groq.com/openai/v1/embeddings
Endpoint de chat completions	https://api.groq.com/openai/v1/chat/completions
Modelo de embeddings	`embed-english-v3.0`
Modelo de chat	`llama3.1:8b` (`llama-3.1-8b-instant`)

Las evidencias de la creacion de la cuenta en GROQ y de la generacion de la API KEY se entregan

como capturas de pantalla y se pueden adjuntar al documento.

> *La configuracion de la API KEY se realiza en la celda 2 del notebook, donde mediante el secret `GROQ_API_KEY` de Google Colab (o variable de entorno) se construye el cliente de GROQ para embeddings y el cliente de chat.*

3. Metodologia del RAG sobre el documento PDF

17. Se carga el **documento PDF en ingles** y se extrae su texto.
18. Se divide el texto en **chunks** de aproximadamente 500 caracteres con solapamiento de 50.
19. Se generan los embeddings de los chunks con el modelo `embed-english-v3.0` de GROQ.
20. La **busqueda densa** recupera los chunks mas similares mediante **similitud coseno**.
21. La **busqueda dispersa** recupera los chunks mas relevantes mediante **BM25**.
22. La **busqueda hibrida** combina densa y dispersa mediante **Reciprocal Rank Fusion (RRF)**.
23. Con el contexto recuperado se construye el prompt y se envia a GROQ para que el modelo `llama3.1:8b` genere la respuesta a la consulta.

4. Resultados del punto 3: comparacion de busquedas con llama3.1:8b

Con el modelo **llama3.1:8b** se ejecutaron las **3 consultas** sobre el documento con el resultado de cada una de las **3 busquedas**: **hibrida**, **densa** y **dispersa**.

4.1 Consultas utilizadas

#	Consulta
1	What is the main topic of this document?
2	What are the key findings or conclusions presented in the text?
3	Summarize the most important points of the document in a short answer.

4.2 Comparacion de resultados

Consulta	Busqueda densa	Busqueda dispersa	Busqueda hibrida
Consulta 1	[respuesta con busqueda densa]	[respuesta con busqueda dispersa]	[respuesta con busqueda hibrida]
Consulta 2	[respuesta con busqueda densa]	[respuesta con busqueda dispersa]	[respuesta con busqueda hibrida]
Consulta 3	[respuesta con busqueda densa]	[respuesta con busqueda dispersa]	[respuesta con busqueda hibrida]

> *Para cada consulta se aplico el modelo **llama3.1:8b** al contexto recuperado por cada busqueda y se comparo la respuesta obtenida.*

5. Resultados del punto 4: nuevo modelo LLM en GROQ

Se identifico otro modelo LLM dentro de la plataforma **GROQ** y se repitio el punto 3 con el nuevo modelo.

Elemento	Detalle
Modelo nuevo	`llama-3.3-70b-versatile`
Tipo de modelo	Chat / texto
Uso	Repetir el punto 3 con la busqueda **hibrida**

Con el nuevo modelo se realizaron las mismas **3 consultas** con la **busqueda hibrida** y se comparamos las respuestas con las obtenidas con **llama3.1:8b**.

Consulta	Respuesta con `llama3.1:8b`	Respuesta con el nuevo modelo
Consulta 1	[respuesta del modelo llama3.1:8b]	[respuesta del nuevo modelo]
Consulta 2	[respuesta del modelo llama3.1:8b]	[respuesta del nuevo modelo]
Consulta 3	[respuesta del modelo llama3.1:8b]	[respuesta del nuevo modelo]

6. Enlaces

Recurso	Enlace
Notebook de Google Colab	https://colab.research.google.com/drive/1iCONlz-J781Rm6JT5QbiTpdc7MojoerN?usp=sharing
Repositorio del proyecto	https://github.com/EnriqueDiaz2/LLM-Rag.git
Plataforma GROQ	https://groq.com/
Documentacion de la API de GROQ	https://console.groq.com/docs

7. Conclusiones personales

En esta practica he podido comprobar que un sistema de **generacion aumentada por recuperacion (RAG)** no solo depende del modelo de lenguaje, sino que la calidad de las respuestas finales depende en gran medida de la calidad de la recuperacion de la informacion.

Me sorprende que con el modelo **llama3.1:8b** de GROQ se puedan generar respuestas coherentes a partir de un documento PDF en ingles vectorizado con el modelo `embed-english-v3.0`.

La busqueda **hibrida** me permitio combinar la busqueda semantica (densa) con la busqueda basada en terminos (dispersa) para recuperar la informacion mas relevante del documento.

En mi opinion, la busqueda hibrida es la estrategia que en general ofrece mejores resultados, ya que complementa la fuerza de ambos metodos.

Con el segundo modelo identificado dentro de GROQ pude observar que la respuesta al usuario cambia con el modelo utilizado, lo que demuestra que el modelo de chat es una parte fundamental del

sistema RAG.

8. Referencias

- GROQ. *Groq Docs*. https://console.groq.com/docs
- GROQ. *Supported Models*. https://console.groq.com/docs/models
- GROQ. *API Reference*. https://console.groq.com/docs/api-reference
- Meta. *Llama 3.1* (modelo `llama3.1:8b`). https://llama.meta.com/
- Meta. *Llama 3.3* (modelo `llama-3.3-70b-versatile`). https://llama.meta.com/
- Krychkovsky, A., et al. *Reciprocal Rank Fusion*. https://arxiv.org/abs/2005.07217